[Nam Le] - Linux OS Overview & Kernel Module/Driver Basics

1. Basic concept of Operating System

1.1 What is Operating System?

A computer system can be devided roughly into 4 components: **hardware**, **operating system**, **application programs**, and a **user**

- hardware: CPU, memory, the I/O devices - provides the basic computing resources for the system
- application programs: (word processors, spreadsheets, compliers, web browsers) - defines the ways in which these resources are used to solve users' computing problems

--> The operating system controls the hardware and **coordinates** its use among the various application programs for the various users.

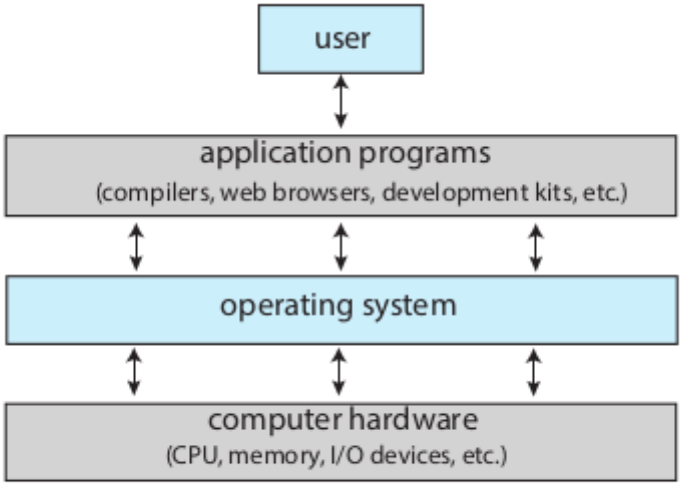


Figure 1.1 Abstract view of the components of a computer system.

Operating system has 2 differents meaning:

- It's the entire package consisting of the central software managing a computer's resources and all of standard software tools (like *command-line interpreters*, *graphical user interfaces*, *file utilities*, *editors*)
- Narrowly, it's refer to central software that manages and allocates computer resources (CPU, RAM, and devices)

> The action of Operating System is same with Kernel purpose, so we will go to Purpose of Kernel if want to know the purpose of OS

1.2 Kernel

Kernel is synonym with **OS**

- It means that **Kernel** is the **core Operating System**
- We can run programs without Kernel, but the presence of a Kernel greatly **simplifies the writing and use of other programs**, increasing the **power** and **flexibility** available to programmers
  --> Kernel can does this by providing a software layer to manage the limited resources of a computer

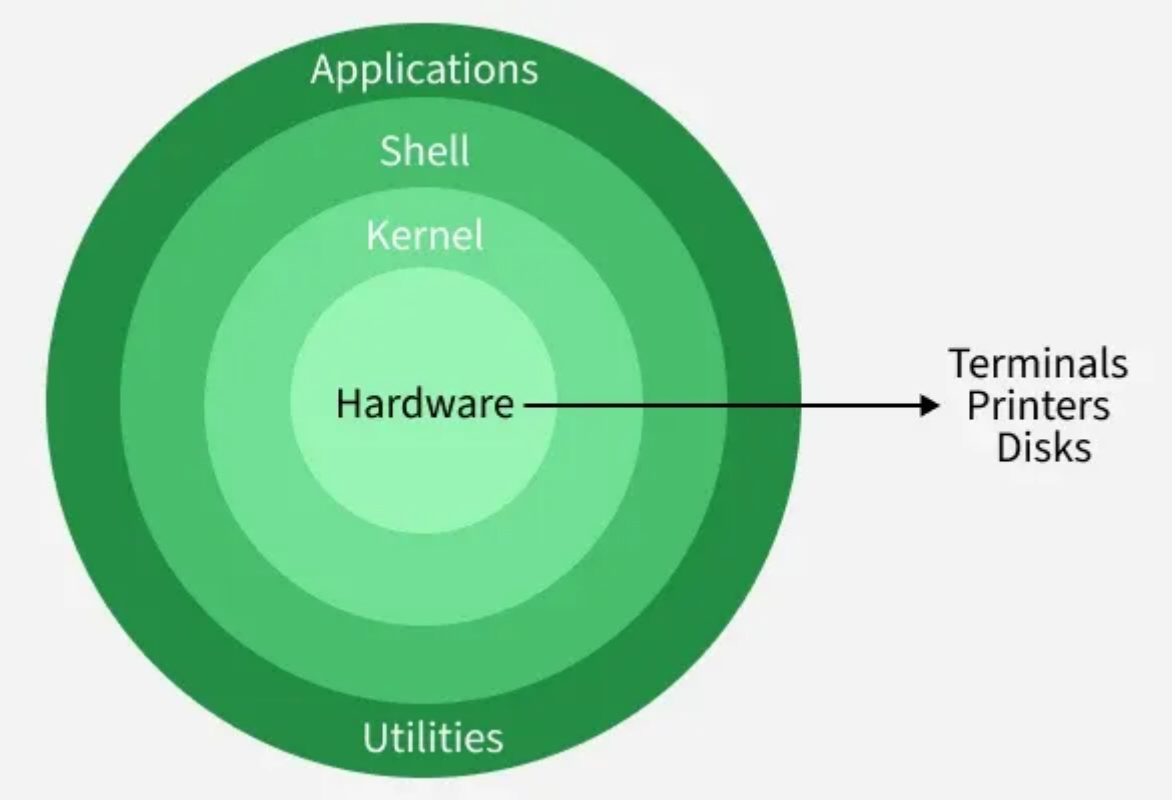
2. Linux Operating System

2.1 Overview

Linux is based on the UNIX operating system, a multi-user and multitasking platform
Linux is free and open-source

- security and stability
- freely use, modify and distribute the operating system

2.2 Linux components



- Hardware: includes physical components like CPU, RAM, storage, and input/output devices.
- Kernel: Core component of Linux that manages hardware resources, processes, and system operations.
- Shell: It is the command-line interface that interprets and executes user commands
- System libraries: provide functions that help applications interact with Linux kernel
- System Utilities: built-in tools used for system manament, configuration, mantainance.

WHAT DOES KERNEL DO?

- **Process** scheduling:
  - Linux is a *preemptive multitasking* operating system
    - Multitasking: means that multiple processes can simultaenously reside in memory and each may receive use of the CPU
    - Preemptive: means that the rules governing which processes receive use of the CPU and for how long are determined by the **kernel process scheduler**
- Creation and termination of **processes**: allocates the memory for programs, and also free if it completely execute
- **Memory** management:
  - physical memory (RAM) remains a limited resource that the kernel must share among processes in an equitable and efficient fashion. Linux employs **virtual memory management** with **2 main advantage**
    - Processes are isolated from one another and from the kernel, so that one process can’t read or modify the memory of another process or the kernel.
    - we have only one memory but can contain one or more than one processes in RAM simultaneously --> This leads to better CPU utilization, since it increases the likelihood that, at any moment in time, there is at least one process that the CPU can execute
- Provision of a **file system**: provide a file system on disk, allowing files to be created, retrived, updated, deleted,...
- Access to **devices**:
  - The devices (mice, monitors, keyboards, disk and tape drives,...) attached to a computer allow communication of information between the computer and **the outside world**
- Networking: The kernel transmits and receives network **packets** on behalf of user processes
- Provision of a System call **Application Programming Interface**:
  - processes can request kernel perform various tasks using kernel entry points, known as **system call**

3. Kernel Space vs User Space

3.1 Concept & Memory

Modern processor architectures typically allow the CPU to operate in at least 2 differents modes: *user mode* and *kernel mode*

Hardware instructions allow switching from one mode to the other.
So, the areas of virtual memory can be marked as being part of *user space* or *kernel space*

- running in user mode: CPU can **access only** memory that it marked as being in **user space**; attempts to access memory in kernel space result in a **hardware exception**
- running in kernel mode: CPU can both access user and kernel space

3.2 The differents about memory when perform a process?

1. In User Space, The memory layout of a process is typically divided into multiple sections

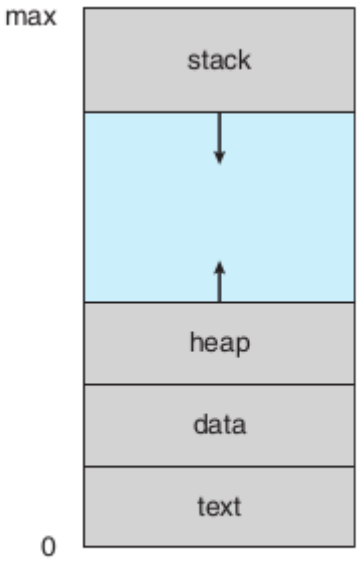
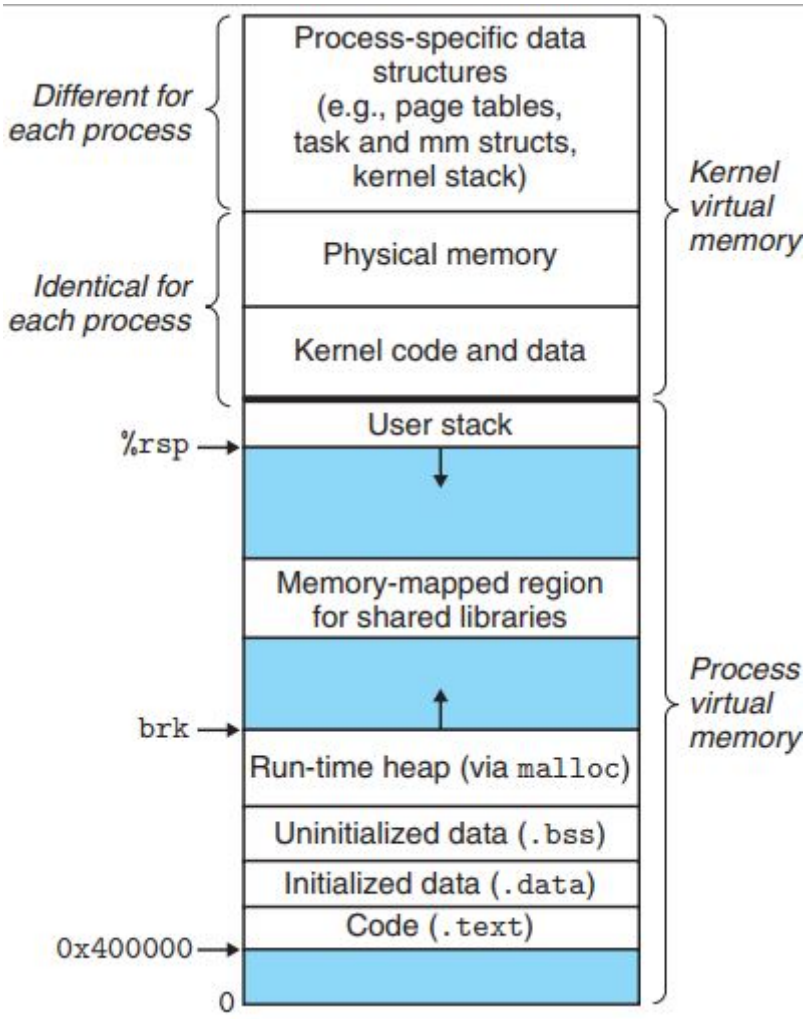


Figure 3.1 Layout of a process in memory.

- text: executable code
- data: golobal variables
- heap: dynamically allocated memory
- Stack: temporary storage when invoking function

2. In Kernel Mode, Linux reserves a part of **Kernel Virtual Memory** for Virtual Address Space of every processes

- This space (The page-table entries) that map to these kernel pages are marked as protected, so the pages are not **visible** or **modifiable** when the processor is running in user mode
- this pre-mapped make more speed whenver process convert from user to kernel (or Conversely)



This picture shows all memory in Virtual Address Space:

- Half of above: Kernel Virtual Address (map to RAM contain **source code** and **data** about **process**)
- Half of bottom: User Virtual Address (map to RAM contain **source code** and **data** of **Operating System**)

4. The interection of Linux OS main components

Linux kernel is **monolithic** architecture, It means that it runs entirely in kernel mode in a single address space
--> They are difficult to implement and extend
--> But, there is very **little overhead** in the **system-call** interface, and **communication** within the kernel is **fast**
NOTE: They do not costly use IPC, They communicate by C system call or shared data instead

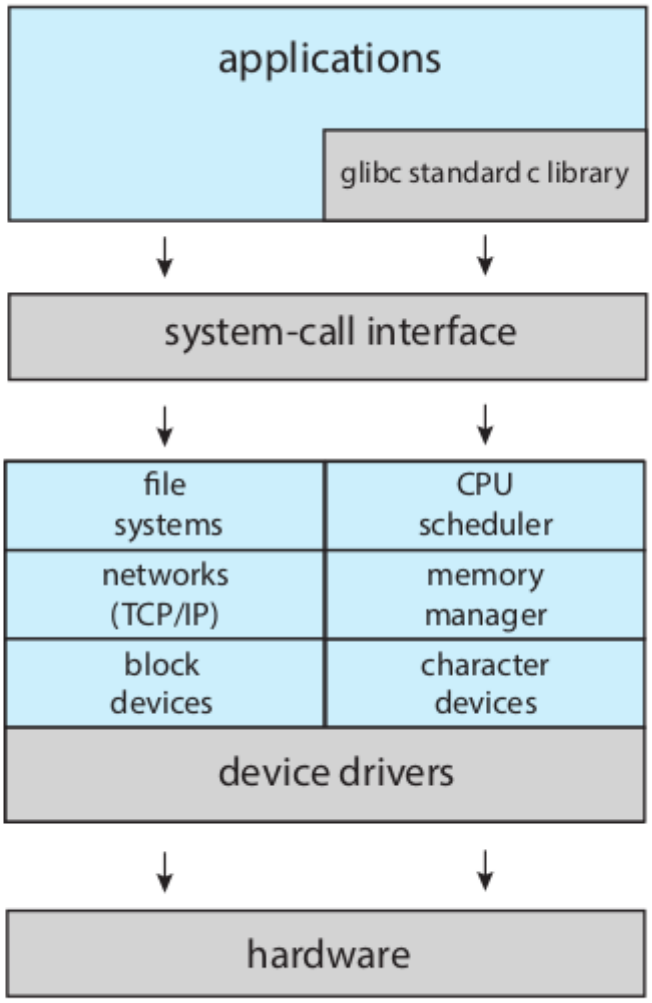


Figure 2.13 Linux system structure.

4.1 Process Management ↔ Memory Management ↔ File System

The process control block is represented by C structure **task_struct**

```
struct task_struct {
  long state;                           /* state of the process */
  struct sched_entity se;               /* scheduling information */
  struct task_struct *parent;           /* this process's parent */
  struct list_head children;            /* this process's children */
  struct files_struct *files;           /* list of open files */
```

```
  struct mm_struct *mm;               /* address space */
};
```

- Process & Memory: `task_struct` has `mm` pointer, point to `struct mm_struct` --> manage the virtual address space of Process
  - When we create a process by `fork()`, the new virtual address area (`vm_area_struct`) will be allocated, use **Copy-on-Write** to share physical memory to Parent Process (for enhance the performance)
- Process & File System: `task_struct` has `files` poniter, point to file description table `struct files_struct`. This table holds `fd` (File Descriptors), each `fd` points to open files `struct file` belong to **VFS (Virtual File System)**
- File System & Memory Management:
  - When a process read/write file system through system call `read()`/`write()` or map file by `mmap()`. They do not read/write directly to **Disk**, instead of it they interact on **Page Cache** on RAM
  - With VFS, Memory use **Slab Allocator** to automatically create and manage cache array for data structure of file system like **inode**, **dentry**, **struct file**

4.2 File System vs Device Drivers

UNIX files can be anything capable of handling the input or output of a stream of data. Device drivers can appear as **files**, and interprocesscommunication channels or network connections also look like **files to the user**

- Block Devices (Hard Disk, SSD): When read/write data from disk, VFS send request to **Request Manager**, **I/O Scheduler**. Then I/O scheduler arrange the requestion before move to **BLOCK DEVICE DRIVER** to control hardware
- Character Device (keyboard, mouse, tty): data is transmited directly to TTY Driver, Line Discripline to Character Device Driver (no pre-process step)

4.3 Network Subsystem ↔ Memory Management ↔ Process ↔ Network Drivers

- Use struct `sk_buff` : Any packet is transmitted through protocol stack and Network Device Driver all packaged in Struct `sk_buff` (**SOCKET BUFFER**)

- When open connection in a process, This Socket port will return fd, then process can interact with network like a file
- Communicate with Network Card: Network Device Driver receive packet from Network Card (NIC), packaged in `sk_buff`

5. Linux Kernel Module

5.1 Concept and Purpose of LKM

Architecture: Although Linux use Monolithic architecture to gain the better performance, this OS still get modularity by LKM
Concept: LKM is source code run in Kernel Mode can be load or unload dynamically into kernel even if system is running or boot, dont need to re-compile all kernel or restart
Purpose: LKMs use to add Device Drivers when use new hardware (like USB Mouse), integrate File System, Network Protocols **GPL** (General Public Lisence): Linux kernel release with GPL, The kernel's module interface allows third parties to write and distribute. Device drivers or file systems that could not be distributed under the GPL

There are 4 components to manage LKM in Linux:

1. **module-management system**: allow modules to be loaded into memory and to communicate with rest of the kernel
2. **Module Loader & Unloader**: utilities in **User Mode** like `insmod`, `rmmod`, `modprobe`, `lsmod` allow load, unload or inspect list of module is running
3. **Driver-Registration System**: After loaded module, notify to kernel about the service which it provide by register to dynamic table of kernel (character/block device, VFS, Network Protocols)
4. **Conflict-Resolution Mechanism**: manage and allocate limited resource (I/O port, Interrupt, DMA) to avoid drivers conflict in share resource

5.2 Write a program to load a module to Linux Kernel

Neccessary **header**:

```
#include <linux/module.h>   // MODULE_*, module_init/exit
#include <linux/kernel.h>   // pr_info, ...
```

```
#include <linux/init.h>     // __init, __exit
```

Entry/exit point:

```
static int __init my_init(void);      /* run when insmod */
static void __exit my_exit(void);       /* run when rmmod */

module_init(my_init);
module_exit(my_exit);
```

The different with UserSpace

- No `libc` : no `printf`, `malloc`
- Dont use floating point
- Small Stack (8 - 16 Kb), note that no big array on Stack
- No dereference directly from userspace pointer, use `copy_from_user()` / `copy_to_user()`
- Error when write module can effect to whole system, must test in VM (Virtual Machine)

Example:

```
/* hello.c */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

static int __init hello_init(void)
{
    pr_info("hello: module loaded\n");
    return 0;
}

static void __exit hello_exit(void)
{
    pr_info("hello: module unloaded\n");
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Nam");
MODULE_DESCRIPTION("Basic hello world kernel module");
```

```
/* Makefile */
obj-m += hello.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

all:
	$(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
	$(MAKE) -C $(KDIR) M=$(PWD) clean
```

Check metadata of kernel object: `modinfo hello.ko`

```
filename:       /home/hoainam0508/Documents/Linux_Kernel_Module/hello.ko
description:    Basic hello world kernel module
author:         Nam
license:        GPL
srcversion:     8EED7D70DF355128794BF57
depends:
retpoline:      Y
name:           hello
vermagic:       6.8.0-138-generic SMP preempt mod_unload modversions
```

```
sudo insmod hello.ko      # load module
lsmod | grep hello        # confirm loaded
dmesg | tail              # see "hello: module loaded"
sudo rmmod hello          # unload module
dmesg | tail              # see "hello: module unloaded"
```

The different in compile between User Mode with Kernel Mode

6. Linux Driver

6.1 Linux Driver seperate to 3 main layers

Block Device Drivers: for Storage Device, allow random access to SSD, CD-ROM, USB
Character Device Drivers: for accessed serially Device, like keyboard, mouse, port
Network Device Drivers: for Network card (Ethernet, Wi-Fi)

Simple driver on Pi4

`sudo apt update && sudo apt upgrade -y`:

- application: Chronium browser, Word Editor, VLC media player, Python Node.js
- System Component: Desktop Invironment, Hardware driver (Wi-Fi, Bluetooth, Sound), Network Tools
- Kernel: Linux kernel, make communication between software vs hardware of Pi

`sudo apt install build-essential raspberrypi-kernel-headers`: for Kernel Header file

Example:

```
/* gpio_driver.h */
#ifndef MY_DRIVER_H
#define MY_DRIVER_H

#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/uaccess.h>
#include <linux/miscdevice.h>
#include <linux/io.h>

#define GPSET0     0x1C
#define GPCLR0     0x28
#define GPIO_BASE  0xFE200000   /* Physical base address of BCM2711 (Pi4) */

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Nam Le");
```

```
MODULE_DESCRIPTION("GPIO Driver for Blinking LED");

static void led_on(unsigned int pin);
static void led_off(unsigned int pin);

static ssize_t driver_proc_write(struct file *file_pointer, const char __user *user_buffer,
size_t len, loff_t *offset);
static int __init gpio_init(void);
static void __exit gpio_exit(void);

#endif
```

```
/* gpio_driver.c */#include "gpio_driver.h"

static void __iomem *gpio;

/* Led on */
static void led_on(unsigned int pin) {
    u32 reg = (pin / 10) * 4;       /* Each register handle 10 pins, each section occupie 4
bytes */
    u32 shift = (pin % 10) * 3;     /* Each pin has 3 bit */
    u32 v = readl(gpio + reg);      /* Save result in virtual variable */

    v &= ~(7 << shift);             /* Set 3 bit in a pin to default (0) */
    v |= (1 << shift);              /* Set 3 bit to 001 --> Output pin*/

    writel(v, gpio + reg);          /* Hard officially choose this pin is OUTPUT */

    writel(1 << pin, gpio + GPSET0); /* Led on */
}

/* Led off */
static void led_off(unsigned int pin) {
    writel(1 << pin, gpio + GPCLR0);
}

static ssize_t driver_proc_write(struct file *file_pointer, const char __user *user_buffer,
size_t len, loff_t *offset) {
    char buf[16];
    unsigned int pin, state;
```

```
    if (len >= sizeof(buf))
        return -EINVAL;
    if (copy_from_user(buf, user_buffer, len))
        return -EFAULT;
    buf[len] = '\0';

    if (sscanf(buf, "%u %u", &pin, &state) != 2 || pin > 27) /* seperate to 2 integer variable
and check */
        return -EINVAL;

    if (state)
        led_on(pin);
    else
        led_off(pin);
    return len;
}

static const struct file_operations fops = {
    .owner = THIS_MODULE,
    .write = driver_proc_write,
};
static struct miscdevice misc = {               /* Register the configration to Linux Kernel
*/
.minor = MISC_DYNAMIC_MINOR,           /* Allocate a Miror Number for this device*/
.name  = "gpio_driver",         /* -> /dev/gpio_simple */
.fops  = &fops,                       /* Link file to function */
};

/* Init driver */
static int __init gpio_init(void)
{
gpio = ioremap(GPIO_BASE, 0x100);        /* Use this address to control Pin/Led */
if (!gpio)
  return -ENOMEM;

return misc_register(&misc);      /* Kernel self-create a device in /dev (and more) --> User
can communicate */
}
```

```
/* Exit driver */
static void __exit gpio_exit(void)
{
misc_deregister(&misc);
iounmap(gpio);
}

module_init(gpio_init);
module_exit(gpio_exit);
```

```
/* Makefile */
obj-m += gpio_driver.o

KDIR := /lib/modules/6.8.0-138-generic/build
PWD := /home/hoainam0508/Documents/Linux_Kernel_Module/gpio_driver

all:
$(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
$(MAKE) -C $(KDIR) M=$(PWD) clean
```

To run on board:

```
make
insmod gpio_driver.ko

echo "17 1" | sudo tee /dev/gpio_simple    /* Turn on */
echo "17 0" | sudo tee /dev/gpio_simple    /* Turn off */
sudo rmmod gpio_simple
```

Try to turn on sequentially

```
/* ring_led.sh */
#!/bin/bash
PINS=(23 22 27 17)
trap 'for p in ${PINS[@]}; do echo "$p 0" > /dev/gpio_driver; done; exit' INT

while true; do
for p in "${PINS[@]}"; do
echo "$p 1" > /dev/gpio_driver
```

```
    sleep 0.2
    echo "$p 0" > /dev/gpio_driver
  done
done
```

Revision #5
Created 2026-10-12 02:31:38 UTC by Lê Hoài Nam
Updated 2026-10-19 10:36:36 UTC by Lê Hoài Nam